COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 009B	**SECTION:**	CPE21S4
DATE PERFORMED:	10/1/26	**DATE SUBMITTED:**	10/1/26
NAME:	Denzen D. Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _10_ Graphs

1. PROCEDURE OUTPUT

ILO A

main.cpp | new.h

Act10p1 > new.h > ...

```
1   #ifndef NEW_H
2   #define NEW_H
3
4   #include <iostream>
5
6   // stores adjacency list items
7   struct adjNode{
8       int val, cost;
9       adjNode* next;
10  };
11
12  // structure to store edges
13  struct graphEdge{
14      int start_ver, end_ver, weight;
15  };
16
```

```cpp
class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head){
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head; // point new node to current head
        return newNode;
    }
    int N; // number of nodes in the graph
    public:
        adjNode **head; // adjacency list as array of pointers
        // constructor
        DiaGraph(graphEdge edges[], int n, int N){
            // allocate new node
            head = new adjNode* [N] ();
            this->N = N;
            // initialize head pointer for all vertices
            for(int i = 0; i < N; ++i){
                head[i] = nullptr;
            }
            // construct directed graph by adding edges to it
            for(unsigned i = 0; i < n; i++){
                int start_ver = edges[i].start_ver;
                int end_ver = edges[i].end_ver;
                int weight = edges[i].weight;
                // insert in the beginning
                adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

                // point head pointer to new node
                head[start_ver] = newNode;
            }
    }
```

```
51      // Destructor
52      ~DiaGraph(){
53          for(int i = 0; i < N; i++){
54              delete[] head[i];
55          delete[] head;
56          }
57      }
58  };
59
60  // print all adjacent vertices of given vertex
61  void display_AdjList(adjNode* ptr, int i){
62      while(ptr != nullptr){
63          std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ")";
64          ptr = ptr->next;
65      }
66      std::cout << std::endl;
67  }
68
69
70  #endif
```

OBSERVATION

- The implementation creates a directed graph consisting of 6 nodes in an adjacency list, and every node outputs its edges as (start_node, end_node, weight). As the edges are added to the front of the list, the edges of a node will be in reverse order to those in the edges array, and so node 0 lists its edge to 2 before its edge to 1. Node 5 does not have any outgoing edges, and hence the corresponding line is blank.

main1.cpp

act10 > main1.cpp > ...

```cpp
#include <iostream>
#include"new.h"



// graph implementation
int main(){
    // graph edges array
    graphEdge edges[] = {
        // (x, y, W)->edge from x to y with weight w
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };
    int N = 6; //Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list represenatation of the graph
    std::cout << "Graph adjacency list " << std::endl << "(start_vertex, end_vertex, weight): " << std::endl;

    for(int i = 0; i < N; i++){
        // display adjacentvertices of vertex i
        display_AdjList(diagraph.head[i], i);
    };
    return 0;
}
```

OBSERVATION

- The main.cpp file makes the edges array and finds out how many edges there are using sizeof, and creates the graph with 6 vertices. After that, a for loop iterates over each vertex, calling the function display_AdjList to print the edges of each vertex. Because this is a directed graph, only one direction of the edge {3, 1, 4} shows up in the output.

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\msys64\ucrt64\bin> C:\Emptech\IGLOSO\act10\main1.exe
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4)(0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

OBSERVATION

- The output provides the adjacency list of the directed graph, where each edge appears as (start_vertex, end_vertex, weight). For vertex 0, the order is (0, 2, 4) followed by (0, 1, 2), since new edges are added to the front of the list. No edges are coming out from vertex 5; hence, there is no output for that vertex. The header line has one typo as well: "end_vertex" instead of "end_vertex."

ILO B1 DFS

DFS.h | newmain.cpp

Act10p1 > ILOB1 > DFS.h > create_reference_graph<T>()

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge{

    size_t src;
    size_t dest;
    T weight;
    // To compaze edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
std :: ostream &operator << (std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){
    os << i << ":\t";
    auto edges = G.outgoing_edges(i);
    for (auto &e : edges){
        os << "{" << e.dest << ": " << e.weight << "}, ";
    }
    os << std :: endl;
    }
    return os;
}

template <typename T>
class Graph{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N){}
    // Return number of vertices in the graph
    auto vertices() const{
        return V;
    }
    // Return all edges in the graph
    auto sedges () const{
        return edge_list;
    }

```

```cpp
void add_edge (Edge<T> &&e){
    // Check if the source and destination vertices are within range
    if (e.src >= 1 && e.src <= V &&
        e.dest >= 1 && e.dest <= V)
        edge_list.emplace_back (e);
    else
        std :: cerr << "Vertex out of bounds" << std :: endl;
}

// Returns all outgoing edges from vertex v
auto outgoing_edges(size_t v) const{
    std :: vector<Edge<T>> edges_from_v;
    for (auto &e : edge_list){
        if (e.src == v)
            edges_from_v.emplace_back(e);
    }
    return edges_from_v;
}

// Overloads the << operator so a graph be written directly to a stream
// Can be used as std :: cout << obj << std :: endl;

template <typename U>
friend std::ostream &operator << (std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

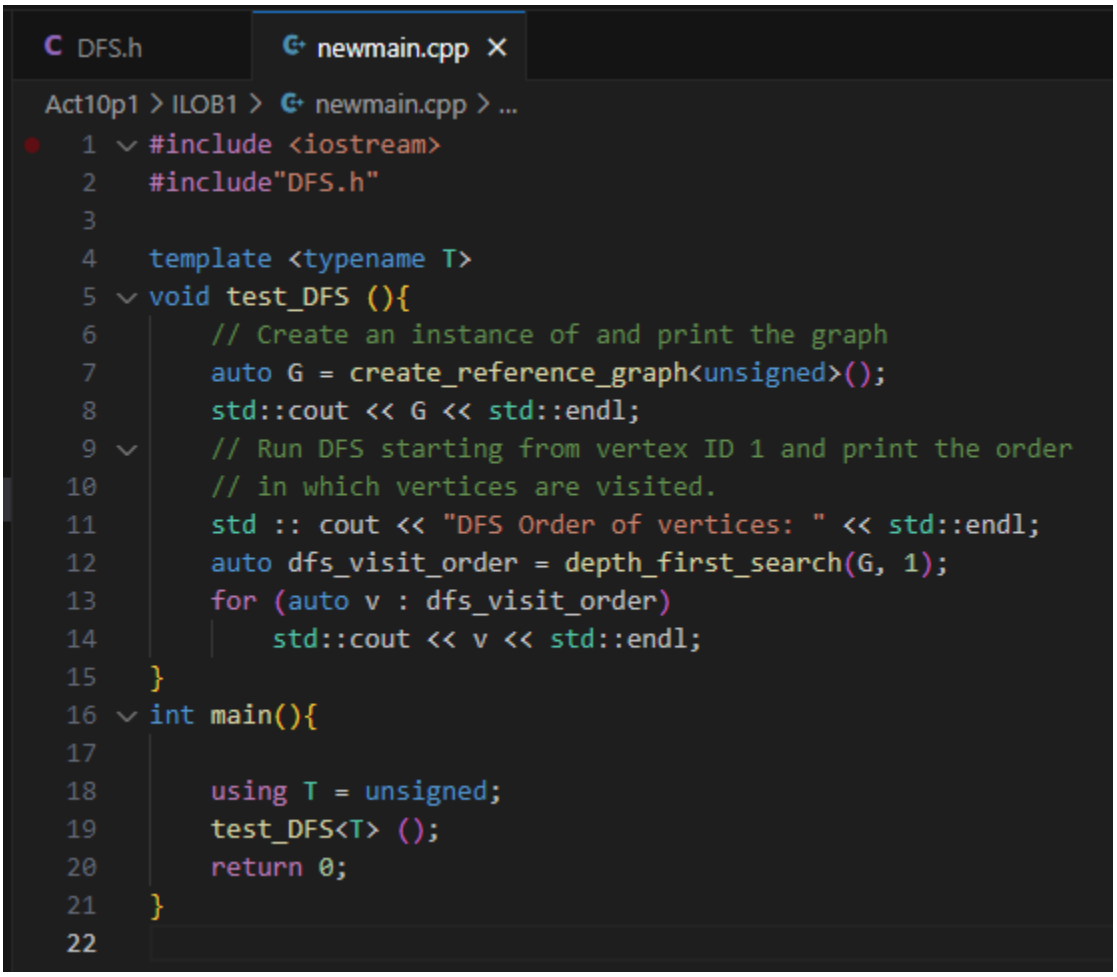
```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest){
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty()){
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find (current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back (current_vertex);
            for (auto e : G.outgoing_edges (current_vertex) ){
                // If the vertex hasn't been visited, insert it in the stack
                if(visited.find(e.dest) == visited.end()){
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges [1] = {{2, 0}, {5, 0}};
    edges [2] = {{1, 0}, {5, 0}, {4, 0}};
    edges [3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges [7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

OBSERVATION

- The DFS.h source code file includes the definition of the Edge struct, the Graph class with the edge list

representation, and the depth_first_search function that implements the DFS algorithm using a stack and a set of already visited vertices in order not to process one vertex more than once. The reference graph consists of 9 vertices and has all edge costs equal to 0 because in DFS edge costs are not required.



OBSERVATION

- newmain.cpp contains DFS.h and utilizes test_DFS to generate the reference graph, display the graph and perform depth first search from vertex 1. The order in which vertices are visited is displayed on separate lines. The only operations carried out by main are setting T as unsigned and calling test_DFS.

```
--pid=Microsoft-MIEngine-Pid-kwe333n3.ymr    --dbg
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

OBSERVATION

- Output shows the edges of vertices from 1 to 8. Vertices traversed in DFS are in the order 1, 5, 8, 6, 7, 3, 4, 2. The next step after visiting vertex 1 is 5 since it is the most recent neighbor that was placed on the stack, and it continues exploring further before returning to other vertices. Vertex 9 does not come into the list or DFS order since there are no edges for it.

ILO B2 BFS

ilobmain.cpp | graph.h

act10 > graph.h > Graph<T> > outgoing_edges(size_t) const

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <cstddef>
#include <iostream>
#include <map>
#include <queue>
#include <set>
#include <string>
#include <vector>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

```

```
21
22      inline bool operator<(const Edge<T> &e) const{
23          return this->weight < e.weight;
24      }
25
26      inline bool operator>(const Edge<T> &e) const{
27          return this->weight > e.weight;
28      }
29  };
30
31  template <typename T>
32  class Graph
33  {
34  public:
35      Graph(size_t N) : V(N) {}
36
37      auto vertices() const{
38          return V;
39      }
40
41      auto &edges() const{
42          return edge_list;
43      }
44
45      void add_edge(Edge<T> &&e){
46          if (e.src >= 1 && e.src <= V &&
47              e.dest >= 1 && e.dest <= V)
48              edge_list.emplace_back(e);
49          else
50              std::cerr << "Vertex out of bounds" << std::endl;
51      }
52
53      auto outgoing_edges(size_t v) const{
54          std::vector<Edge<T>> edges_from_v;
55          for (auto &e : edge_list){
56              if (e.src == v){
57                  edges_from_v.emplace_back(e);
58              }
59          }
```

```
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }
    return os;
}

#endif
```

OBSERVATION

- The outgoing_edges function loops through all of the edges every time it is called, making its complexity O(E), irrespective of the number of edges of any vertex. Since the function is called once for every vertex visited by BFS, it results in the complexity being closer to O(V·E) than O(V+E).

ilobmain.cpp × graph.h

act10 > ilobmain.cpp > breadth_first_search<T>(const Graph<T>&, size_t)

```cpp
#include "graph.h"

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```cpp

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1);

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }

    return visit_order;
}

```

```
52
53  template <typename T>
54  void test_BFS()
55  {
56      // Create an instance of and print the graph
57      auto G = create_reference_graph<unsigned>();
58      std::cout << G << std::endl;
59
60      // Run BFS starting from vertex ID 1 and print the order
61      // in which vertices are visited.
62      std::cout << "BFS Order of vertices: " << std::endl;
63      auto bfs_visit_order = breadth_first_search(G, 1);
64      for (auto v : bfs_visit_order)
65          std::cout << v << std::endl;
66  }
67
68  int main()
69  {
70      using T = unsigned;
71      test_BFS<T>();
72      return 0;
73  }
```

OBSERVATION

- When BFS verifies visited vertices, it does this when vertices are dequeued, not when they are enqueued, meaning that the same vertex can be enqueued multiple times. In the example graph, the vertex 4 gets enqueued five times (by 2, 3, 5, 6, and 8), but only once is this vertex visited.

```
PS C:\msys64\ucrt64\bin> C:\Emptech\IGLOSO\act10\ilobmain.exe
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\msys64\ucrt64\bin>
```

OBSERVATION

- The order of the BFS algorithm for 1, 2, 5, 4, 8, 3, 6, 7 arranges the vertices depending on how close they are to vertex 1 (vertices 2 and 5, followed by 4 and 8, followed by 3 and 6, and vertex 7). The order within each group depends on the order in which the vertices have been inserted. Vertex 4 precedes vertex 8 since it is reached via vertex 2, which is dequeued before vertex 5.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- Depth-First Search (DFS). The way the problem goes is that the individual starts at one point, moves further down to an end and then returns to explore other ways out of the same node. That is precisely what happens when we use DFS as it makes use of a stack to keep track of which way to return to. The BFS method will not work here because it covers all the nodes first before moving further down.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- While DFS for graphs is depth-first traversals, which include pre-order traversal (Root, Left, Right), in-trees. In-order and post-order are depth-first traversals too. While the level order traversal is the BFS in trees.

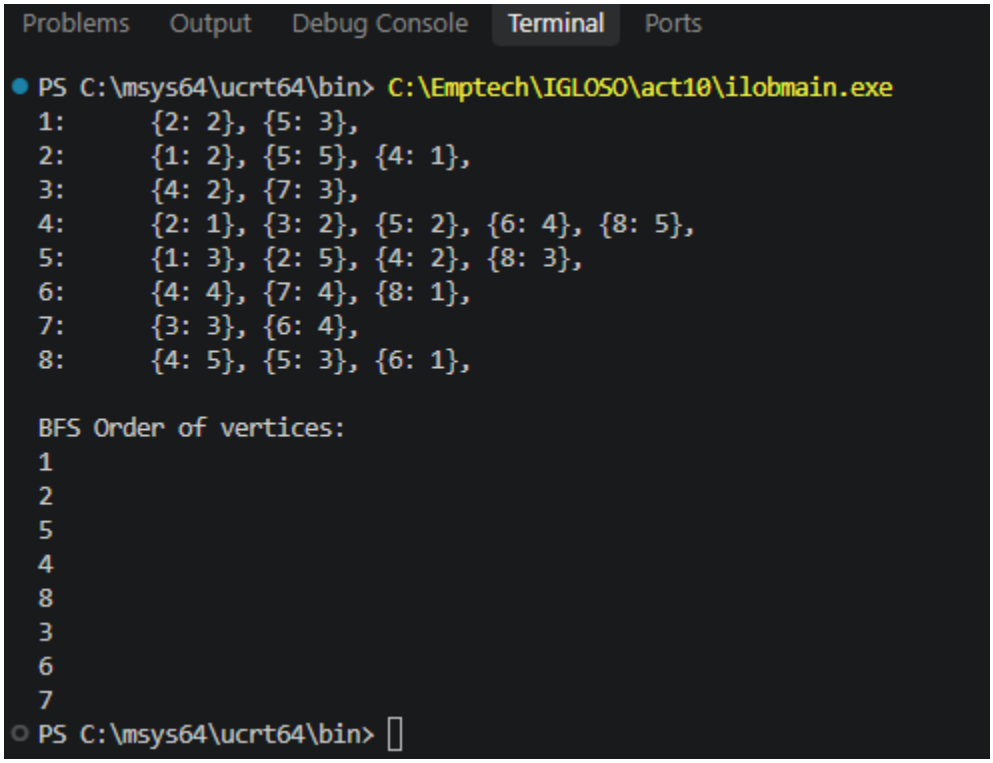
DFS (pre-order): A -> B -> D -> E -> C
BFS (level-order): A -> B -> C -> D -> E

```
DFS(G, start):
    create empty stack S and empty set visited
    push start onto S
    while S is not empty:
      v = pop S
       if v is not in visited:
          add v to visited and record v
          for each neighbor w of v:
              push w onto S
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t start)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(start);
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                stack.push(e.dest);
        }
    }
    return visit_order;
}
```



OBSERVATION

- In the above reference graph, this DFS will traverse vertices in the following sequence: 1, 5, 8, 6, 7, 3, 4, 2. On the other hand, BFS produces this result: 1, 2, 5, 4, 8, 3, 6, 7. DFS will explore vertices deeper first and visit vertex 2 last although vertex 2 is adjacent to vertex 1 because the neighbor which was pushed more recently (5) than vertex 2 is popped first from the stack.

3. In the performed code, what data structure is used to implement the Breadth First Search?

- In the Breadth First Search implementation within the code, the queue used is a std::queue<size_t>. The traversal through the graph is done on a FIFO basis with the help of a std::set and std::vector to keep track of the vertices.

4. How many times can a node be visited in the BFS?

- There will be no repeat visitation of any node using the BFS algorithm because the visited set will prevent this from happening. However, when we consider the code that we used, there can be several push operations on any particular node, one per each adjacent edge that connects to the node. As an illustration, node 4 is pushed five times, through nodes 2, 3, 5, 6, and 8. Once it is dequeued the first time, it passes the check and is not processed anymore.

CONCLUSION

- From this activity, I have learned how graphs can be constructed using an edge list, and how BFS traverses them using a queue. The algorithm was easy to understand after defining the Graph and Edge classes and produced the expected output since vertex 9 never appeared since it had no edges

connected to it. In the extension section, I was able to compare BFS with DFS, where DFS is recursive and dives deep first using a stack, whereas BFS uses breadth-first search by traversing level by level. Overall, I believe I performed well, although I must still improve my DFS programming skills.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and google**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Used for grammar correction and searching for code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.